I want to start this report off by saying this: I failed to find three SQLi vulnerabilities in the Hackazon mobile API. I was able to find two SQLi vulnerabilities and one XSS vulnerability. It took me three days and around 24 hours to find these three vulnerabilities. I'm not sure if I was doing something wrong, or if it had something to do with the way the Hackazon mobile app worked. I apologize for not reaching the goal, but I needed to stop looking for my own sanity. I don't say all this as an excuse. I say it to let you know I tried my best to find these vulnerabilities.

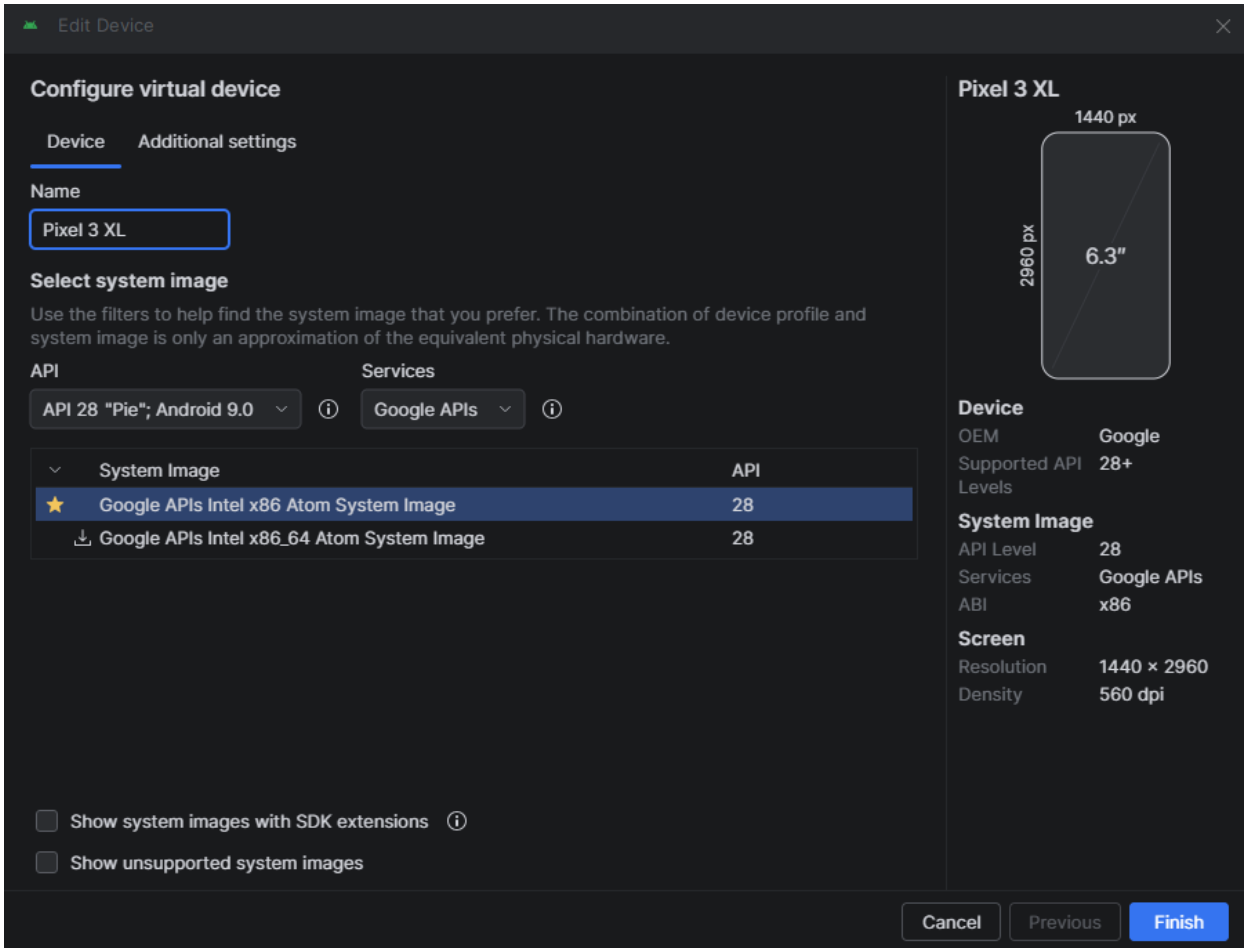


Fig 1: Virtual Android config.

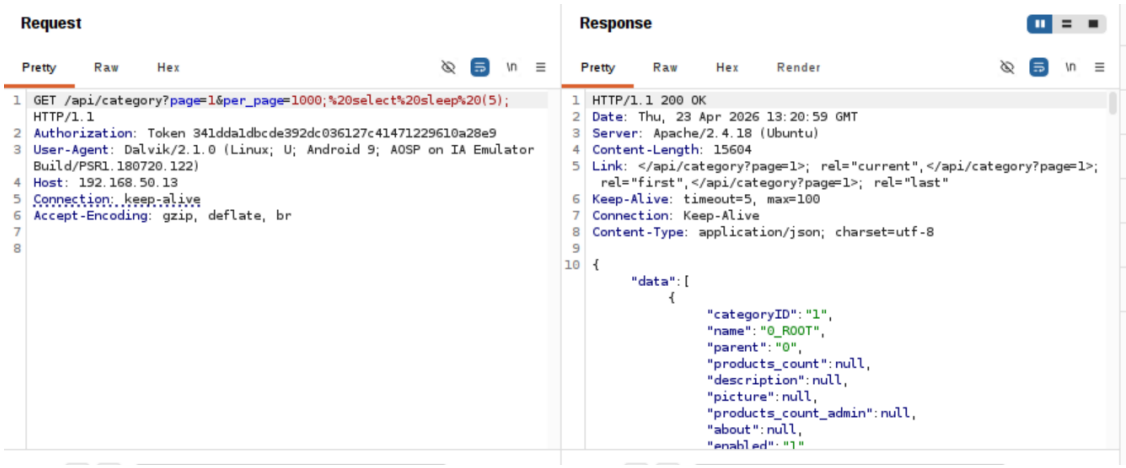


Fig 2: The first SQL vulnerability.

This first SQLi vulnerability is hard to see. It is a blind SQL vulnerability. The SQLi vulnerability is in the *per_page* parameter. By imputing:*1000;%20select%20sleep%20(5);* we cause the response to take an extra five seconds to respond. This tells us that the *per_page* parameter is vulnerable to SQL injection.

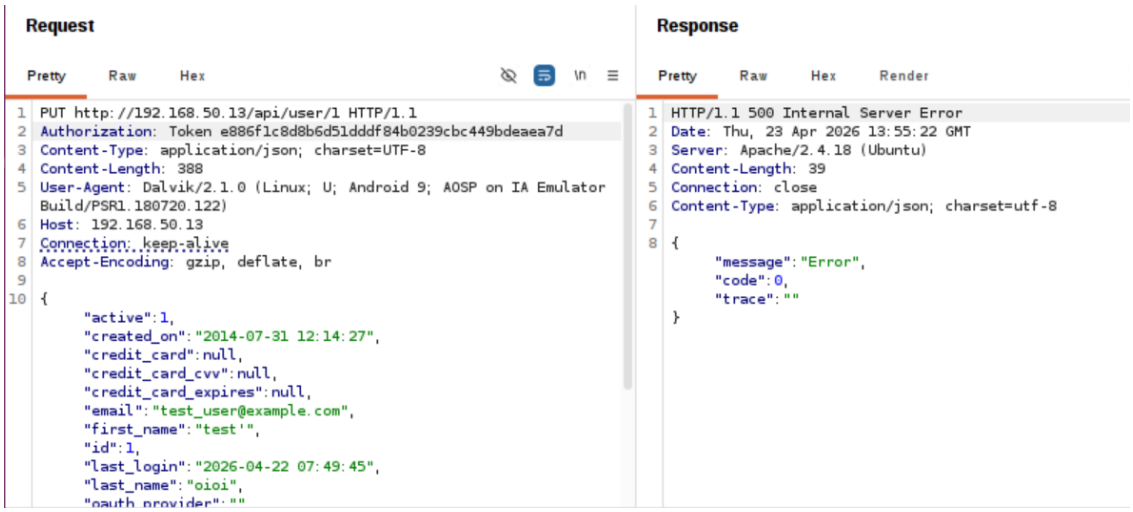


Fig 3: Response to test'

The second vulnerability is also a blind SQL vulnerability. We can change the first_name parameter to test'. This gives a 500 response or an internal server error. However, when we

change the first_name parameter to test' ' it returns with a 200 OK response. This tells us that this parameter is vulnerable to SQL injections.

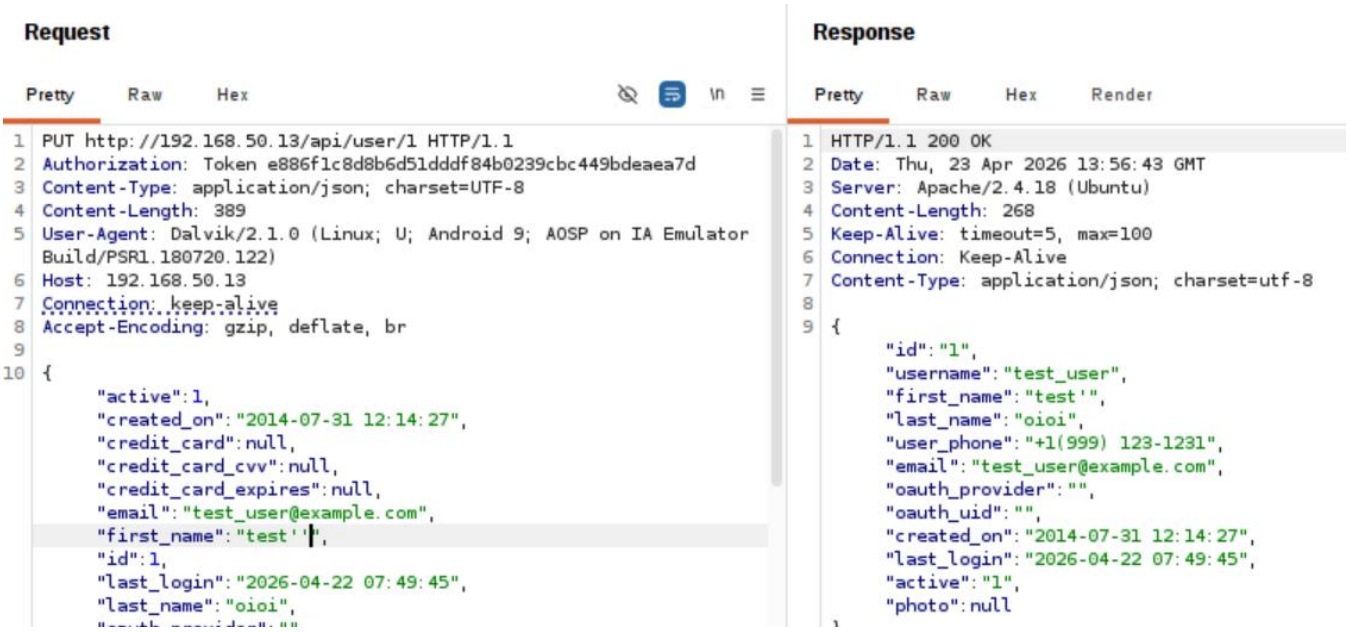


Fig 4: Response to test' '.

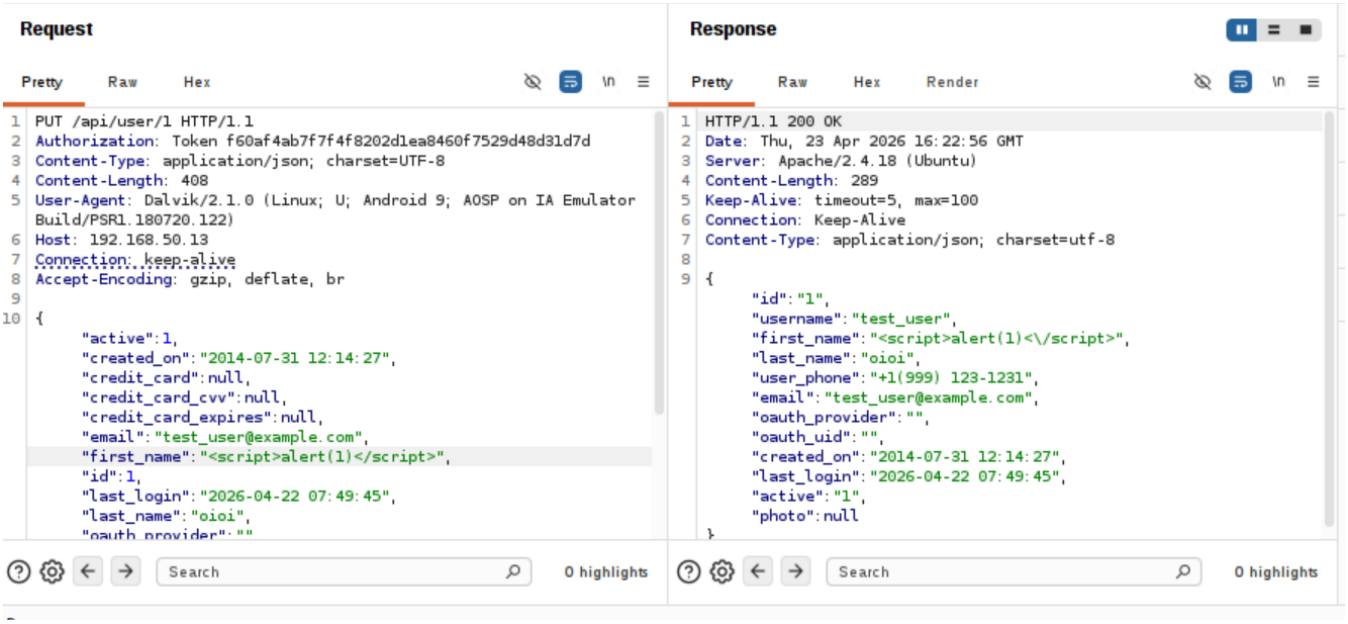


Fig 5: Using first_name to XSS.

The third vulnerability I discovered wasn't SQL. It is an XSS. I found this vulnerability to be interesting. We start by updating the first_name parameter to an XSS script. As we learned from the previous vulnerability the first_name parameter is vulnerable to attacks. The script I used in the parameter will create a popup that says 1. This is the interesting part. This attack doesn't work on the mobile app. It happens when the user logs in onto the website. This can be seen in Fig 6.

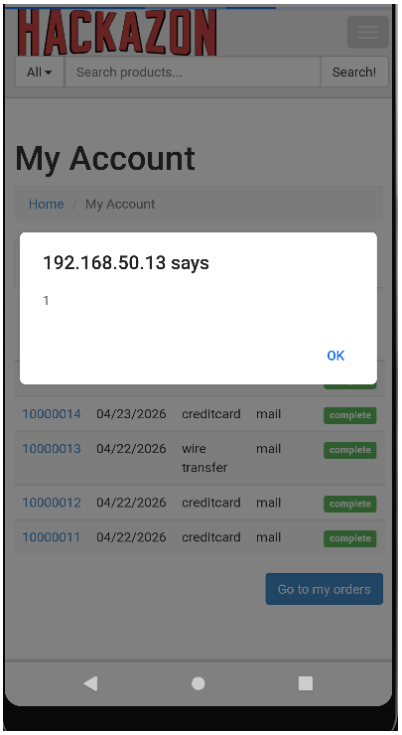


Fig 6: XSS successful.

What do these vulnerabilities mean? Why do they matter? SQL is used to steal data from the backend of a database. But sometimes the database doesn't respond with information or errors. That is where we use blind SQL. Blind SQLs are used to see if the server is vulnerable to SQL but doesn't give a normal response. (Blind SQL Injection | OWASP Foundation, n.d.). In

the first vulnerability we used a time-based SQL. We forced the response to be delayed. That told us the sever was vulnerable to SQL, it just doesn’t reply as expected. The second SQL vulnerability is also blind. We don’t get information in the responses; we use the responses as the information. We started with test' and it responded with a server error. We tried again with test' ' and received an okay from the server. This was a sign of SQL; we changed how the server acted by crashing it.

The final vulnerability was an XSS vulnerability. Cross Site Scripting (XSS) attacks can do a lot of different things. Though they are usually focused on stealing login information, cookies and session tokens. (Cross Site Scripting (XSS) | OWASP Foundation, n.d.-a). We were able to confirm this attack by logging into the web application and receiving the popup. These vulnerabilities show the threat to personal information that exists in the world today.

References:

Blind SQL Injection | OWASP Foundation. (n.d.-b). https://owasp.org/www-community/attacks/Blind_SQL_Injection

Cross Site Scripting (XSS) | OWASP Foundation. (n.d.-b). https://owasp.org/www-community/attacks/xss/